

Robotic Navigation HW1 Report

A* and RRT* Algorithm Implementation

114062633 / 黃薪橙

March 16, 2026

1 A* 演算法實作邏輯(A-Star Algorithm)

A* 演算法透過結合實際已走過的代價 (g-score) 與到達終點的預估代價 (h-score, 啟發函數) 引導搜尋方向。在 `a_star_implementation.py` 中的實作如下：

1.1 preloop(self)

- **初始化資料結構**：利用 `heapq` 建立優先權佇列(`self.queue`) 來隨時取得總代價(`f_score`) 最低的節點。同時建立計數器(`self.counter`) 避免代價相同時發生比較報錯的問題。
- **起點設置**：紀錄起點的實際代價 $g = 0$ ，並計算起點距離終點的直線距離作為啟發預估代價，接著將起點推入佇列，並建立 `self.visited_nodes` 記錄已探索的節點集合。

1.2 step(self)

- **選擇最佳節點**：每次從佇列最前端取出 `f_score` 最低的節點(`current_node`)。若該節點已在 `visited_nodes` 集合中則跳過不處理，反之則將其標記為已拜訪。
- **終點判定**：若當前節點距離終點小於 `goal_threshold` 容忍誤差，則判定抵達終點，設定終點的父節點並觸發 `self.is_done.set()` 完成搜尋。
- **擴展鄰居節點**：遍歷周遭八宮格（或相鄰）且尚未拜訪過的鄰居節點。當計算出經過當前節點前往鄰居的實際成本(`tentative_g`) 較小時，則更新該鄰居的 g 值及父節點指標，並計算其新的 `f_score` 將之推入優先權佇列中等待日後擴展。

1.3 postloop(self)

演算法收斂並抵達終點後，利用 `collect_path(self.goal_node)` 沿著父節點的指標不斷往回追溯至起點，生成並回傳完整的最佳路徑供視覺化介面繪製。

2 RRT* 演算法實作邏輯(RRT-Star)

RRT* 是基於隨機採樣的路徑規劃演算法，在建立亂數生長樹的同時，透過重新連接(Rewire) 機制斬斷次優路徑並接上更優的路徑，以保證演算法隨著採樣次數增加漸進收斂至最佳解。`rrt_star_implementation.py` 的實作如下：

2.1 preloop(self)

建立`self.visited_nodes` 集合代表當前的生長樹，將起點加入樹中並設定成本為0。同時初始化`self.children` 字典結構，用以記錄各個節點間「父→ 集合(子)」的關係，這是為了在後續Rewire 時能夠有效地遞迴更新子樹成本。

2.2 step(self)

- **隨機採樣與延伸(Sample & Steer)**：在空間中隨機採樣一個目標點，並從樹中尋找距離最近的節點(`nearest_node`)。沿此方向按固定步長(`step_size`) 延伸產生一個新的`new_node`。
- **碰撞檢查(Collision Detection)**：確保留新節點在邊界內，且連接`nearest_node` 到`new_node` 的線段不與任何障礙物發生碰撞。
- **連接最佳父節點(Choose Parent)**：在給定搜尋半徑(`search_radius`) 中找出樹上所有的相鄰節點。計算若從這些相鄰節點出發連接至`new_node`，哪一條路徑的總抵達成本最低。選擇總成本最低的節點作為其父節點，並將新節點加入樹結構。
- **優化重接(Rewire)**：再次遍歷所有相鄰節點。若該相鄰節點改變心意，改為從剛剛產生的`new_node` 連接過去能產生更低的成本，則將其父節點改為`new_node`。同時呼叫`self.propagate_cost()` 神經遞迴網路，將降低的成本往下廣播。
- **抵達目標**：若`new_node` 距離終點足夠近且無碰撞，且預期到達終點的成本比目前的歷史最佳成本(`best_goal_cost`) 還低，則更新最佳成本並儲存關聯，通知系統完成搜尋。

2.3 propagate_cost(self, node)

深度優先搜尋(DFS) 輔助函數。每當Rewire 階段改變父子關係使成本調降時，該函數會透過`self.children` 關係鏈，把降低的成本差異一路傳遞更新給該分支底下的所有衍生子節點。

2.4 postloop(self)

搜尋結束後，藉由從終點的父節點指標一路回溯到起點，取得能避開障礙物並朝向最佳化收斂的平滑軌跡，並回傳提供前端框架更新繪圖畫面。